POLITECHNIKA GDAŃSKA

WYDZIAŁ ELEKTRONIKI, TELEKOMUNIKACJI I INFORMATYKI

Wprowadzenie do języka Ada

Magdalena Godlewska

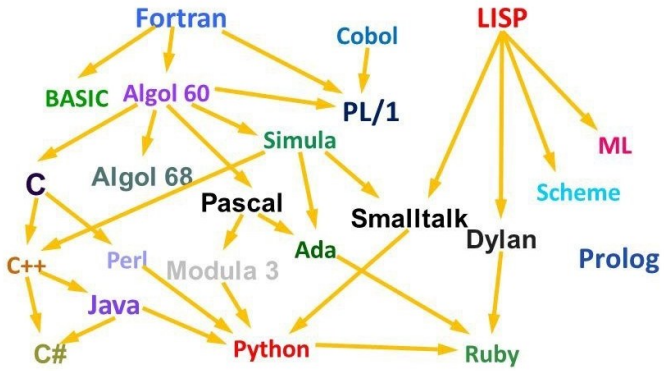
1

Krótko o historii

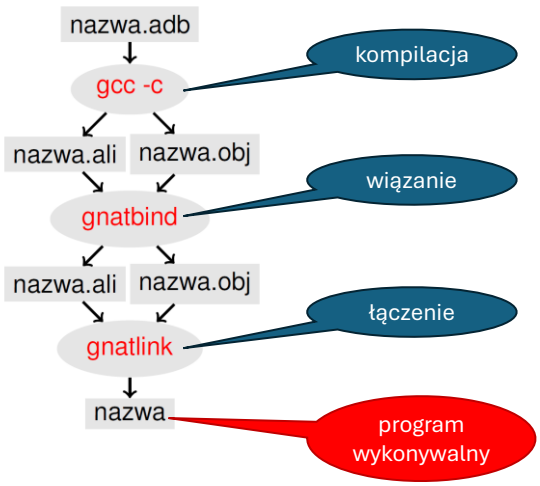
- Współbieżny, strukturalny, kompilowany, imperatywny, statycznie typowany język programowania opracowany przez Jean Ichbiaha i zespół z CII Honeywell Bull w latach 70. XX wieku.
- Wygrał konkurs zorganizowany przez Departament Obrony USA (DoD), pokonując 19 innych projektów.
- Wcześniej DoD USA dostrzegł problem wykorzystywania dużej liczby różnych języków programowania używanych dla wewnętrznych systemów wbudowanych. Wiele z nich było przestarzałych, związanych ze sprzętem i nie wspierało bezpiecznego programowania modułowego..
- Nazwa języka, nadana przez DoD, pochodzi od imienia Augusty Ady Lovelace (maszyna analityczna – pierwsza programistka).
- W 1987 roku DoD wydał zarządzenie wymagające użycia Ady w każdym projekcie programistycznym, gdzie nowy kod stanowił więcej niż 30% ogólnego wyniku. Reguła została zniesiona 10 lat później.
- Z uwagi na bezpieczeństwo, Ada jest używana w aplikacjach wojskowych oraz projektach komercyjnych, gdzie błąd programistyczny może mieć kosztowne konsekwencje. Np. kontrola lotów, awionika samolotów, satelity i inne systemy kosmiczne, automatyczne systemy transportowe czy bankowość (Fly-by-wire w samolocie Boeing 777 zostało w całości napisane w Adzie).

2

Krótko o historii



Kompilacja dla piszących „z palca”



lub

```
~$ gnatmake nazwa.adb
gcc-4.4 -c nazwa.adb
gnatbind -x nazwa.ali
gnatlink nazwa.ali
```

Kompilacja dla piszących „z palca" – program modułowy

producent.ads
```
package Producent is
  procedure Produkuj(...);
  procedure Wyslij(...);
end Producent;
```

konsument.ads
```
package Konsument is
  procedure Przyjmij(...);
  Procedure Konsumuj(...);
end Konsument;
```

producent.adb
```
package body Producent is
...
end Producent;
```

konsument.adb
```
package body Konsument is
...
end Konsument;
```

gmain.adb
```
with...; use...;
procedure Gmain is
...
end Gmain;
```

kompilacja

```
gcc -c gmain.adb
gcc -c producent.adb
gcc -c konsument.adb
```

wiązanie i łączenie

```
gnatbind gmain
gnatlink gmain
```

lub

```
gnatmake gmain
```


Dla preferujących IDE



GNAT Studio | AdaCore
Releases · AdaCore/gnatstudio (github.com)

Dostęp 30.09.2024

Program startowy: simulation.adb
oraz instrukcja uruchomienia go w GPS: GNAT2021_instrukcja.pdf
znajdują się na e-nauczaniu

Najprostszy program w Adzie – procedura główna

```
with Ada.Text_IO;
use Ada.Text_IO;

procedure Main is
begin
   Put_Line("Hello Ada!");
end Main;
```


Podprogramy

```
function nazwa(parametry) return typ is
   deklaracje
begin
   . . .
   return wyrażenie;
end nazwa;
```

```
procedure nazwa(parametry) is
   deklaracje
begin
   . . .
end nazwa;
```

Parametry:

function f(X, Y : FLOAT; N : INTEGER := 0) **return** FLOAT **is**. . .

rodzaje:

- wejściowe (też domyślnie) **in**
- wyjściowe **out** (tylko w procedurze)
- **in out**
- przez wskaźnik **access**

jeśli nie ma parametrów, opuszczamy nawiasy

Funkcje (a także operatory) można przeciążać.

Zadania

.. czyli fragmenty kodu, które wykonywane są asynchronicznie (tj. niezależnie od siebie), ale od czasu do czasu komunikują się wymieniając pomiędzy sobą dane.

Program zaprojektowany jako wielozadaniowy:

- jest łatwiejszy w konserwacji (małe porcje kodu)
- umożliwia tworzenie systemów ze stopniową degradacją, czyli jeżeli w pewnym zadaniu pojawi się błąd to system jako taki nie przestanie działać, a co najwyżej straci nieco na swojej funkcjonalności
- po błędzie zadania, może ono zostać wznowione, przez co system odzyska swoją funkcjonalność
- na odpowiednim sprzęcie zadania mogą być wykonywane równolegle (współbieżnie), przez co program będzie wykonywany szybciej

9

Najprostsze zadanie

```
task Robot;

task body Robot is
begin
  Put_Line("Robot rozpoczyna pracę");

  for I in 1 .. 5 loop
    Put_Line("Robot pracuje");
    delay 1.0;
  end loop;

  Put_Line("Robot kończy pracę");
end Robot;
```

Procedura główna oraz zadanie Robot wykonują się współbieżnie.

10

Zadania a spotkania

Deklaracja wraz z możliwymi wejściami

```
task type Aabb is
    entry APrint;
    entry BPrint;
end Aabb;
```

Definicja i obsługa spotkań

```
task body Aabb is
  begin
     accept APrint do
       Put_Line(" AAA ");
     end APrint;

     accept BPrint do
       Put_Line(" BBB ");
     end BPrint;
  end Aabb;
```


Zadania – przykład

```
task type Aaa is
    entry Start;
end Aaa;

task type Bbb is
    entry Start;
end Bbb;

task type Aabb is
    entry APrint;
    entry BPrint;
end Aabb;

A: Aaa;
B: Bbb;
AB: Aabb;
```

```
task body Aaa is
   subtype Aaa_Time_Range
      is Integer range 3 .. 6;
   package Random_Aaa
      is new
Ada.Numerics.Discrete_Random(Aaa_Time_Range);
   G: Random_Aaa.Generator;
   Random_Time: Duration;

 begin
   accept Start do
     Random_Aaa.Reset(G);
   end Start;
   Put_Line("Aaa start");
   loop
     Random_Time :=
Duration(Random_Aaa.Random(G));
    delay Random_Time;

     AB.APrint;

   end loop;
 end Aaa;
```

```
task body Aabb is
  begin
    Put_Line("Aabb started");
    loop

      accept APrint do
        Put_Line(" AAA ");
      end APrint;

      accept BPrint do
        Put_Line(" BBB „);
      end BPrint;

    end loop;
  end Aabb;


begin
  A.Start;
  B.Start;
```

```
Aabb started
Aaa start
Bbb start
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
```

Spotkania

W Adzie istnieje możliwość bezpośredniej komunikacji zadań za pomocą mechanizmu nazywanego spotkaniem (fr. rendez-vous).

Mechanizm ten polega na tym, że zadanie udostępniające usługę i zadanie chcące z tej usługi skorzystać czekają w pewnym punkcie programu na siebie i następnie wykonywany jest fragment kodu zadania realizującego to spotkanie.

Oba zadania są zajęte realizowanych spotkaniem i pozostają niedostępne dla innych zadań – dochodzi do synchronizacji

Po wykonaniu tego fragmentu kodu zadania biorące udział w spotkaniu kontynuują swoje działanie asynchronicznie.

Spotkania selektywne oczekiwane (select ... or)

```
task body Aabb is
  begin
    Put_Line("Aabb
started");
    loop
     select
       accept APrint do
         Put_Line(" AAA ");
       end APrint;
     or
       accept BPrint do
         Put_Line(" BBB „);
       end BPrint;
     end select
    end loop;
  end Aabb;
```

```
Aabb started
Aaa start
Bbb start
 AAA
 BBB
 BBB
 AAA
 BBB
 AAA
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
 BBB
 AAA
 BBB
 AAA
 BBB
 AAA
 BBB
```

Spotkania selektywne z budzikiem/przeterminowaniem (select ... or delay)

```
task body Aabb is
  begin
    Put_Line("Aabb
started");
    loop
     select
      accept APrint do
        Put_Line(" AAA ");
      end APrint;
     or
      delay 5.0;
      accept BPrint do
        Put_Line(" BBB „);
      end BPrint;
     end select
    end loop;
  end Aabb;
```

```
Aabb started
Aaa start
Bbb start
 BBB
 AAA
 AAA
 BBB
 AAA
 AAA
 AAA
 AAA
 BBB
 AAA
 AAA
 BBB
 AAA
 AAA
 AAA
 AAA
 BBB
 AAA
 AAA
 BBB
 AAA
 BBB
```

15

Spotkania selektywne z warunkowym wywołaniem (select ... else)

```
task body Aabb is
  begin
    Put_Line("Aabb
started");
    loop
     select
      accept APrint do
        Put_Line(" AAA ");
      end APrint;
     else
        delay 1.0;
        Put_Line (" NUDZE
SIE ");
     end select
    end loop;
  end Aabb;
```

```
Aabb started
Aaa start
Bbb start
 NUDZE SIE
 NUDZE SIE
 NUDZE SIE
 NUDZE SIE
 AAA
 NUDZE SIE
 NUDZE SIE
 NUDZE SIE
 NUDZE SIE
 AAA
 NUDZE SIE
 NUDZE SIE
 NUDZE SIE
 AAA
 NUDZE SIE
 NUDZE SIE
 NUDZE SIE
 NUDZE SIE
 NUDZE SIE
 AAA
 NUDZE SIE
```

16

Spotkania selektywne z asynchroniczną zmianą wątku sterowania (select ... then abort)

```
task body Aabb is
  begin
    Put_Line("Aabb
started");
    loop
     select
      accept APrint do
        Put_Line(" AAA ");
      end APrint;
     or
      accept BPrint do
        Put_Line(" BBB „);
      end BPrint;
     end select
    end loop;
  end Aabb;
```

```
task body Aaa is
....
    a : String (1 .. 3);
    b : Integer;
  begin
....
    loop
      Random_Time := Duration
(Random_Aaa.Random (G));
      select
        delay Random_Time;
        AB.APrint;
      then abort
        Put_Line ("teraz nic nie robie, nacisnij
przycisk");
        Get_Line (a, b);
      end select;
    end loop;
  end Aaa;
```

```
Aabb started
Aaa start
teraz nic nie robie, nacisnij przycisk
Bbb start
 BBB
 BBB
 BBB
 BBB

 AAA
teraz nic nie robie, nacisnij przycisk

teraz nic nie robie, nacisnij przycisk

teraz nic nie robie, nacisnij przycisk

teraz nic nie robie, nacisnij przycisk

teraz nic nie robie, nacisnij przycisk
 BBB

 AAA
teraz nic nie robie, nacisnij przycisk
 BBB
 BBB
```

17

Dozory

Instrukcja **accept** może mieć rozszerzoną składnię używaną w ramach instrukcji **select**, służącą do okresowego blokowania wejścia – nazywa się ją dozorem

when Warunek **=> accept** Nazwa(parametry) **do**

```
task body Aabb is
   subtype Aabb_Range is Integer range 1 .. 6;
   package Random_aabb is new Ada.Numerics.Discrete_Random (Aabb_Range);
   G : Random_aabb.Generator;
   dice: Integer;
 begin
   Put_Line (ESC & "[91m" & "Aabb started" & ESC & "[0m");
   Random_aabb.Reset (G);
   loop
     dice := Random_aabb.Random(G);
     Put_Line("Dice: "  & Integer'Image(dice));

     select
       when dice=6 => accept APrint do
         Put_Line (ESC & "[95m" & " AAA " & ESC & "[0m");
       end APrint;

     or
       accept BPrint do ....
```

```
Aabb started
Dice:  2
Aaa start
Bbb start
 BBB
Dice:  4
 BBB
Dice:  5
 BBB
Dice:  2
 BBB
Dice:  1
 BBB
Dice:  2
 BBB
Dice:  1
 BBB
Dice:  5
 BBB
Dice:  6
 AAA
Dice:  2
 BBB
```

18

Mini-samouczek i skrypt Ady

Na e-nauczaniu znajdują się pomoce:

- samouczek.pdf – przygotowany przeze mnie skrypt zawierający elementy Ady, które uznałam za szczególnie przydatne przy realizacji projektu
- Ada_95_Skrypt_Wyd_II.pdf – skrypt do Ady pobrany z Internetu

19

Zasady realizacji projektu

1. Należy dobrać się w 2-3 osobowe zespoły (najlepiej 3) i zapisać się do jednej z grup na e-nauczaniu
2. Realizujecie zadanie w grupie, możecie skorzystać z konsultacji
3. Jedna z osób w grupie wygrywa na e-nauczanie rozwiązanie zadania do **25.10.2026**
4. Na e-nauczaniu będzie zamieszczony harmonogram oddawania projektów. W wyznaczonym terminie proszę całą grupkę o przyjście chwilę przed czasem i demonstrację rozwiązania (ok 10 min).

20